

Практическое занятие 2. Линейные регистры сдвига

Учебные вопросы:

1. Принципы построения линейных регистров сдвига
2. Свойства выходных последовательностей линейных регистров сдвига

Задание 1. Реализация простого LFSR

Цель:

Научиться реализовывать простой линейный регистр сдвига с заданным многочленом обратной связи и начальным состоянием.

Условие:

Дан регистр длины 4 бит с многочленом обратной связи $(x^4 + x + 1)$ (т.е. обратная связь берется с первого и четвертого битов). Начальное состояние регистра — $[1, 0, 0, 0]$.

Напишите программу, которая генерирует последовательность бит длиной 15, используя данный LFSR.

Объяснение решения

- LFSR — это последовательный сдвиг регистра, где новый бит формируется как XOR выбранных битов текущего состояния (тех, что указаны в многочлене обратной связи).
- Для многочлена $(x^4 + x + 1)$ обратная связь берётся с 4-го и 1-го битов.
- После вычисления нового бита регистр сдвигается вправо, а новый бит появляется слева.
- Максимальный период для 4-битного LFSR — $(2^4 - 1 = 15)$.

Пример кода на Python

```
def lfsr(taps, state, length):
    """
    taps: список позиций обратной связи (например, [4,1])
    state: начальное состояние (список битов)
    length: длина генерируемой последовательности
    """
    n = len(state)
    output = []
    for _ in range(length):
        output.append(state[-1]) # выходной бит — последний в регистре
        # вычисляем новый бит как XOR битов на позициях taps
        new_bit = 0
        for t in taps:
            new_bit ^= state[-t]
        # сдвигаем регистр вправо и вставляем новый бит слева
        state = [new_bit] + state[:-1]
    return output

if __name__ == "__main__":
    taps = [4, 1]
    init_state = [1, 0, 0, 0]
    seq_length = 15
    sequence = lfsr(taps, init_state, seq_length)
```

```
print("Сгенерированная последовательность:")
print(sequence)
```

Задание 2. Определение периода LFSR

Цель:

Понять, как определить период последовательности, генерируемой LFSR, и что влияет на его максимальность.

Условие:

Для LFSR из Задания 1 определите период сгенерированной последовательности (сколько уникальных состояний регистра до повторения начального).

Объяснение решения

- Период — количество шагов, после которого состояние регистра повторяется.
- Начальное состояние не должно быть нулевым.
- Максимально возможный период для n -битного регистра — $(2^n - 1)$.
- Нужно хранить состояния регистра и отслеживать повтор.

Пример кода на Python

```
def find_period(taps, init_state):
    state = init_state.copy()
    initial = tuple(state)
    count = 0
    seen_states = set()
    while True:
        count += 1
        new_bit = 0
        for t in taps:
            new_bit ^= state[-t]
        state = [new_bit] + state[:-1]
        state_tuple = tuple(state)
        if state_tuple == initial:
            return count
        if state_tuple in seen_states:
            # Если встретили другое повторение — период меньше
            return count
        seen_states.add(state_tuple)

if __name__ == "__main__":
    taps = [4, 1]
    init_state = [1, 0, 0, 0]
    period = find_period(taps, init_state)
    print(f"Период LFSR: {period}")
```

Задание 3. Изучение влияния многочлена обратной связи на период LFSR

Цель:

Исследовать, как выбор многочлена обратной связи влияет на период и свойства генерируемой последовательности.

Условие:

Для 4-битного регистра сравните периоды LFSR с разными многочленами обратной связи:

- $(x^4 + x + 1)$ (т.е. taps = [4,1])
- $(x^4 + x^3 + 1)$ (taps = [4,3])
- $(x^4 + x^3 + x^2 + x + 1)$ (taps = [4,3,2,1])

Определите для каждого периода.

Объяснение решения

- Примитивные многочлены дают максимальный период ($2^n - 1$).
- Другие многочлены дают меньший период.
- Сравниваем результаты, чтобы увидеть влияние полинома.

Пример кода на Python

```
def find_period(taps, init_state):
    state = init_state.copy()
    initial = tuple(state)
    count = 0
    while True:
        count += 1
        new_bit = 0
        for t in taps:
            new_bit ^= state[-t]
        state = [new_bit] + state[:-1]
        if tuple(state) == initial:
            return count

if __name__ == "__main__":
    init_state = [1, 0, 0, 0]
    polynomials = {
        'x^4 + x + 1': [4, 1],
        'x^4 + x^3 + 1': [4, 3],
        'x^4 + x^3 + x^2 + x + 1': [4, 3, 2, 1]
    }

    for name, taps in polynomials.items():
        period = find_period(taps, init_state)
        print(f"Период для {name}: {period}")
```

Задание 4. Использование LFSR для генерации псевдослучайных чисел

Цель:

Реализовать генератор псевдослучайных чисел на базе LFSR и проверить его свойства.

Условие:

Используйте LFSR с многочленом ($x^5 + x^3 + 1$) и начальным состоянием [1,0,0,0,0], чтобы сгенерировать последовательность длиной 31 бит. Выведите последовательность и оцените распределение единиц и нулей.

Объяснение решения

- LFSR с примитивным многочленом длины 5 даёт период 31.
- Генерация последовательности — просто повтор LFSR.
- Распределение битов близко к равномерному.

Пример кода на Python

```
def lfsr(taps, state, length):
    n = len(state)
    output = []
    for _ in range(length):
        output.append(state[-1])
        new_bit = 0
        for t in taps:
            new_bit ^= state[-t]
        state = [new_bit] + state[:-1]
    return output

if __name__ == "__main__":
    taps = [5, 3]
    init_state = [1, 0, 0, 0, 0]
    length = 31
    seq = lfsr(taps, init_state, length)
    print(«Последовательность LFSR:»)
    print(seq)
    print(f»Количество единиц: {seq.count(1)}»)
    print(f»Количество нулей: {seq.count(0)}»)
```

Задания для самостоятельного выполнения

Задание 5. Реализация LFSR с другим многочленом и начальным состоянием

Условие:

Реализуйте LFSR длины 5 с многочленом обратной связи ($x^5 + x^2 + 1$) (т.е. taps = [5, 2]) и начальным состоянием [0, 1, 0, 0, 1]. Сгенерируйте последовательность длиной 31 бит.

Подсказка:

- Используйте код из Задания 1 как шаблон.
- Обратите внимание на позиции taps и начальное состояние.

Задание 6. Определение периода LFSR с разными начальными состояниями

Условие:

Для LFSR с многочленом ($x^4 + x + 1$) (taps = [4, 1]) найдите период для следующих начальных состояний:

- [1, 0, 0, 0]
- [0, 1, 0, 0]
- [1, 1, 1, 1]

Сравните полученные периоды и сделайте выводы.

Подсказка:

- Используйте функцию определения периода из Задания 2.
- Помните, что начальное состояние не должно быть нулевым.

Задание 7. Исследование влияния многочлена обратной связи на период для 3-битного LFSR

Условие:

Для регистра длины 3 сравните периоды LFSR с многочленами:

- ($x^3 + x + 1$) (taps = [3, 1])
- ($x^3 + x^2 + 1$) (taps = [3, 2])

Начальное состояние — [1, 0, 0].

Подсказка:

- Используйте функцию определения периода.
- Посмотрите, какой многочлен даёт максимальный период (7 для 3-битного регистра).

Задание 8. Генерация псевдослучайных чисел с LFSR и анализ распределения

Условие:

Используя LFSR с многочленом $(x^6 + x + 1)$ ($\text{taps} = [6, 1]$) и начальным состоянием $[1, 0, 0, 0, 0, 0]$, сгенерируйте последовательность длиной 63 бита. Подсчитайте количество единиц и нулей и постройте гистограмму распределения (можно с помощью matplotlib).

Подсказка:

- Код генерации аналогичен Заданию 4.
- Для построения гистограммы используйте:

```
import matplotlib.pyplot as plt
```

```
counts = [seq.count(0), seq.count(1)]  
plt.bar(['0', '1'], counts)  
plt.show()
```

Задание 9. Реализация LFSR с обратной связью по нескольким битам и визуализация состояния

Условие:

Реализуйте LFSR длины 4 с многочленом $(x^4 + x^3 + 1)$ ($\text{taps} = [4, 3]$) и начальным состоянием $[1, 1, 0, 0]$. Выведите в консоль состояние регистра на каждом шаге, а также итоговую последовательность длиной 15.

Подсказка:

- Выводите список бит после каждого сдвига.
- Используйте цикл с печатью.

Тест

1) Что такое линейный регистр сдвига (LFSR)?

- A) Регистр сдвига, в котором следующий бит вычисляется как линейная комбинация предыдущих битов по модулю 2
B) Регистр, который сдвигает биты и добавляет случайное значение в конец
C) Регистр, который хранит только нули и единицы без сдвига
D) Регистр, работающий на основе нелинейных функций

2) Какой максимальный период может иметь n -битный LFSR при использовании примитивного многочлена обратной связи?

- A) (n)
B) (2^n)
C) $(2^n - 1)$
D) $(2^{\{n-1\}})$

3) Для многочлена обратной связи $(x^4 + x + 1)$ какие биты используются для вычисления нового бита?

- A) 4-й и 1-й
- B) 3-й и 2-й
- C) 2-й и 1-й
- D) 4-й и 3-й

4) Что происходит с состоянием регистра после вычисления нового бита?

- A) Регистр сдвигается влево, новый бит добавляется справа
- B) Регистр сдвигается вправо, новый бит добавляется слева
- C) Состояние регистра не меняется
- D) Регистр очищается

5) Какое начальное состояние регистра недопустимо для корректной работы LFSR?

- A) Все биты равны 1
- B) Все биты равны 0
- C) Произвольная последовательность бит
- D) Только старший бит равен 1

6) Что означает, что многочлен обратной связи является примитивным?

- A) Он имеет степень 1
- B) Он порождает максимальный период LFSR
- C) Он содержит все степени от 0 до n
- D) Он не содержит свободного члена

7) Как рассчитывается новый бит в LFSR?

- A) Логическое И (AND) выбранных битов
- B) Логическое ИЛИ (OR) выбранных битов
- C) Исключающее ИЛИ (XOR) выбранных битов
- D) Сложение по модулю 10

8) Какой период будет у 3-битного LFSR с примитивным многочленом?

- A) 3
- B) 7
- C) 8
- D) 15

9) Чем полезны LFSR в практике?

- A) Для хранения больших чисел
- B) Для генерации псевдослучайных чисел и кодирования
- C) Для сжатия данных
- D) Для увеличения скорости процессора

10) Что произойдет, если использовать нелинейную функцию вместо XOR для обратной связи в регистре сдвига?

- A) Получится классический LFSR
- B) Последовательность станет более предсказуемой
- C) Получится нелинейный регистр сдвига (NLFSR) с другими свойствами
- D) Регистр перестанет работать